AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection

C082-Krisha Chawan, C109-Manav Gangar, C116-Jude Dsouza, C108-Neel Vora

Under the guidance of Prof. Sofia Francis

1. Abstract / Introduction / Motivation

Software vulnerabilities create security and reliability risks. Conventional Static Application Security Testing (SAST) tools can produce false positives and miss complex vulnerability patterns, while machine-learning code classifiers add contextual analysis of source code. We propose an AI-assisted Security Copilot that integrates static analysis, CodeBERT-based vulnerability classification, Retrieval-Augmented Generation (RAG), remediation guidance and post-repair verification. The classifier is trained and evaluated on the PyCode-Vul Python dataset. Fine-tuned CodeBERT achieves 96.88% accuracy, 95.76% precision, 97.38% recall, 96.56% F1-score and 99.33% ROC-AUC.

4. Methodology: Multi-Stage Security Copilot Workflow

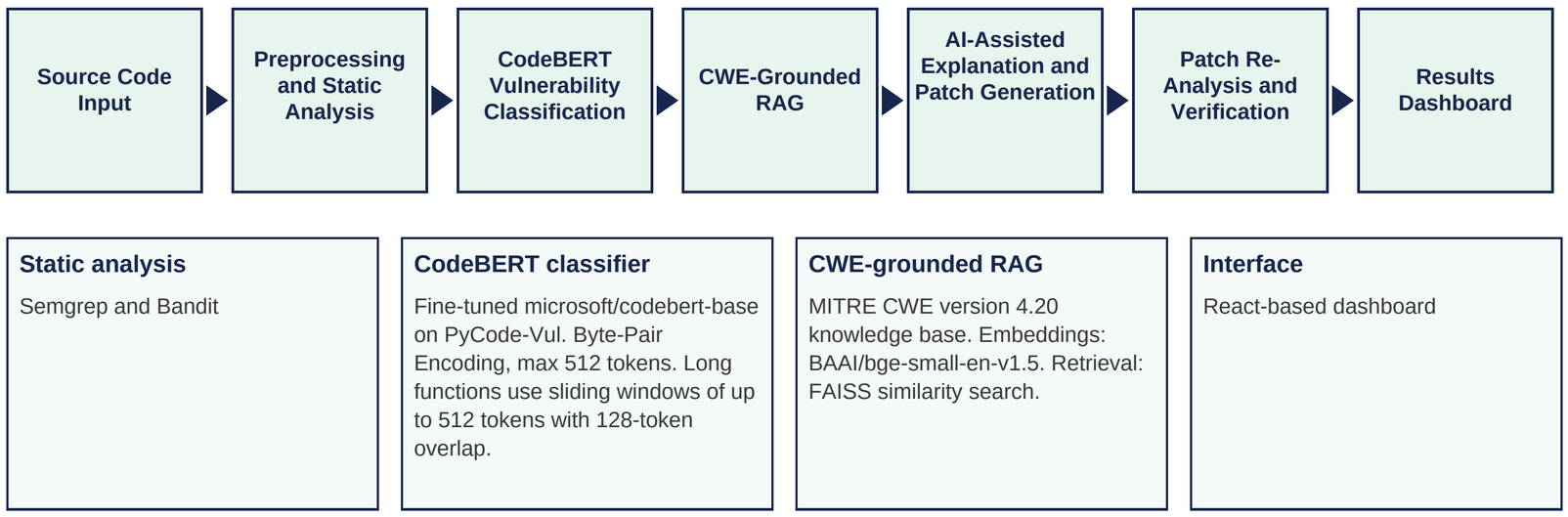


Fig. 1. System architecture of the proposed AI-assisted Security Copilot. The classifier performs binary vulnerability prediction; generated patches are re-analyzed rather than assumed secure.

2. Literature Review / Background

Rule-based vulnerability detection

Tools such as Semgrep, Bandit and CodeQL provide rule-based detection but can suffer from false positives and incomplete coverage.

Transformer-based vulnerability classification

Models such as CodeBERT [1] and GraphCodeBERT [2] learn contextual representations of source code and support vulnerability classification.

RAG-assisted remediation and verification

Retrieval-augmented systems provide security-specific context, while independent verification is needed to assess whether generated patches actually resolve vulnerabilities.

Research gap: this work brings these capabilities together in a unified, Python-focused security-analysis workflow.

3. Objectives

1. Develop a source-code vulnerability classifier by fine-tuning CodeBERT.
2. Integrate rule-based static analysis with contextual vulnerability classification.
3. Incorporate CWE-grounded retrieval to support explanations and remediation recommendations.
4. Establish a repair-verification workflow that re-analyzes generated patches and reports the verification outcome.

Dataset and Training Summary

- 17,676 functions after preprocessing
- 53,073 generated code chunks
- 10 training epochs; validation F1 used for model selection
- Random seed: 42
- Separate training, validation and held-out test sets

The classifier performs binary vulnerability prediction; the full workflow and tool stack are shown in Fig. 1.

5. Results and Discussion

Table 1. CodeBERT classification performance

Metric	CodeBERT
Accuracy	96.88%
Precision	95.76%
Recall	97.38%
F1-score	96.56%
ROC-AUC	99.33%
Average Precision	99.31%

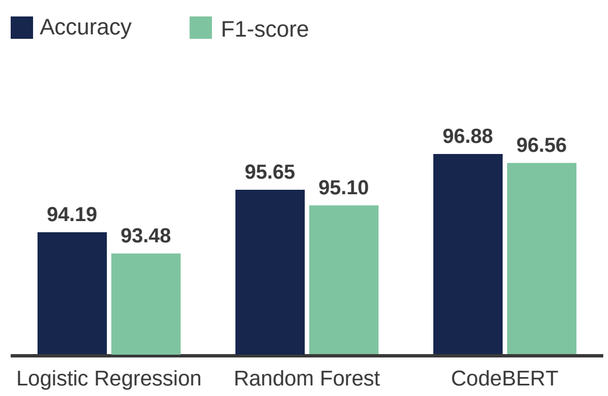
Table 2. Model comparison

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	94.19%	94.23%	92.75%	93.48%	98.68%
Random Forest	95.65%	96.35%	93.88%	95.10%	99.23%
CodeBERT	96.88%	95.76%	97.38%	96.56%	99.33%

Table 3. Ablation study

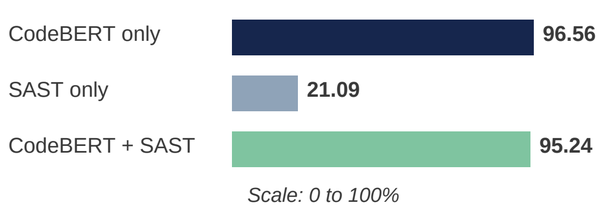
Configuration	Accuracy	Precision	Recall	F1-score	ROC-AUC
CodeBERT only	96.88%	95.76%	97.38%	96.56%	99.33%
SAST only	59.65%	68.68%	12.00%	21.09%	55.26%
CodeBERT + SAST	95.65%	93.55%	97.00%	95.24%	97.58%

Accuracy and F1-score by model



Y-axis starts at 90%

Ablation study: F1-score (%)



Scale: 0 to 100%

Key findings

- CodeBERT achieved the strongest overall classification performance among the evaluated configurations.
- Recall of 97.38% on the held-out test set, with 42 false negatives versus 98 for Random Forest and 116 for Logistic Regression.
- CodeBERT + SAST reached 97.00% recall but did not outperform CodeBERT alone on overall metrics; SAST still contributes rule-based findings.
- CWE-grounded RAG supports explanations and remediation guidance; no accuracy gain is claimed for it.

6. Conclusion

We propose a multi-stage Security Copilot for Python code that combines static analysis, fine-tuned CodeBERT classification, CWE-grounded RAG and patch re-analysis. CodeBERT delivers the strongest classification results reported here; end-to-end repair effectiveness requires separate evaluation.

7. References

[1] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," Findings of EMNLP, 2020, doi: 10.18653/v1/2020.findings-emnlp.139.
[2] D. Guo et al., "GraphCodeBERT: Pre-training Code Representations with Data Flow," ICLR, 2021.
[3] MITRE Corporation, "Common Weakness Enumeration (CWE), version 4.20," https://cwe.mitre.org.
[4] [Insert PyCode-Vul dataset citation from the manuscript bibliography]
[5] [Insert retrieval-augmented vulnerability analysis reference from the manuscript bibliography]
[6] [Insert automated patch generation and verification reference from the manuscript bibliography]

BTI Computer Engineering | Mukesh Patel School of Technology Management and Engineering, SVKM's NMIMS, Mumbai

SVKM'S NMIMS Deemed to be UNIVERSITY
MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING ™